Rising Waters

Project Description

Introduction

Floods are among the most devastating natural disasters, causing significant damage to life, property, agriculture, and infrastructure. Accurate flood prediction enables authorities and communities to take preventive measures, reducing the impact of flooding. Traditional flood forecasting methods often require extensive manual analysis and may not provide timely predictions.

The Rising Waters – AI-Based Flood Prediction System is a machine learning-based web application developed to predict the possibility of floods using rainfall data. The system analyzes annual and seasonal rainfall values and uses a trained Random Forest Machine Learning model to determine whether flood conditions are likely to occur.

The application is developed using the Flask web framework with HTML, CSS, and JavaScript for the frontend. Users can easily enter rainfall values through a web interface, and the application instantly displays the prediction result. The project has been deployed on the Render cloud platform, making it accessible through any web browser without requiring local installation.

The primary objective of the project is to demonstrate how Artificial Intelligence and Machine Learning can assist in disaster management by providing quick and reliable flood predictions.

Project Scenarios

Scenario 1: High Rainfall Prediction

A disaster management officer enters annual and seasonal rainfall values collected from a region experiencing heavy monsoon rainfall. The application processes the data using the trained Random Forest model and predicts a **Flood Chance**, allowing authorities to initiate early warning and emergency preparedness measures.

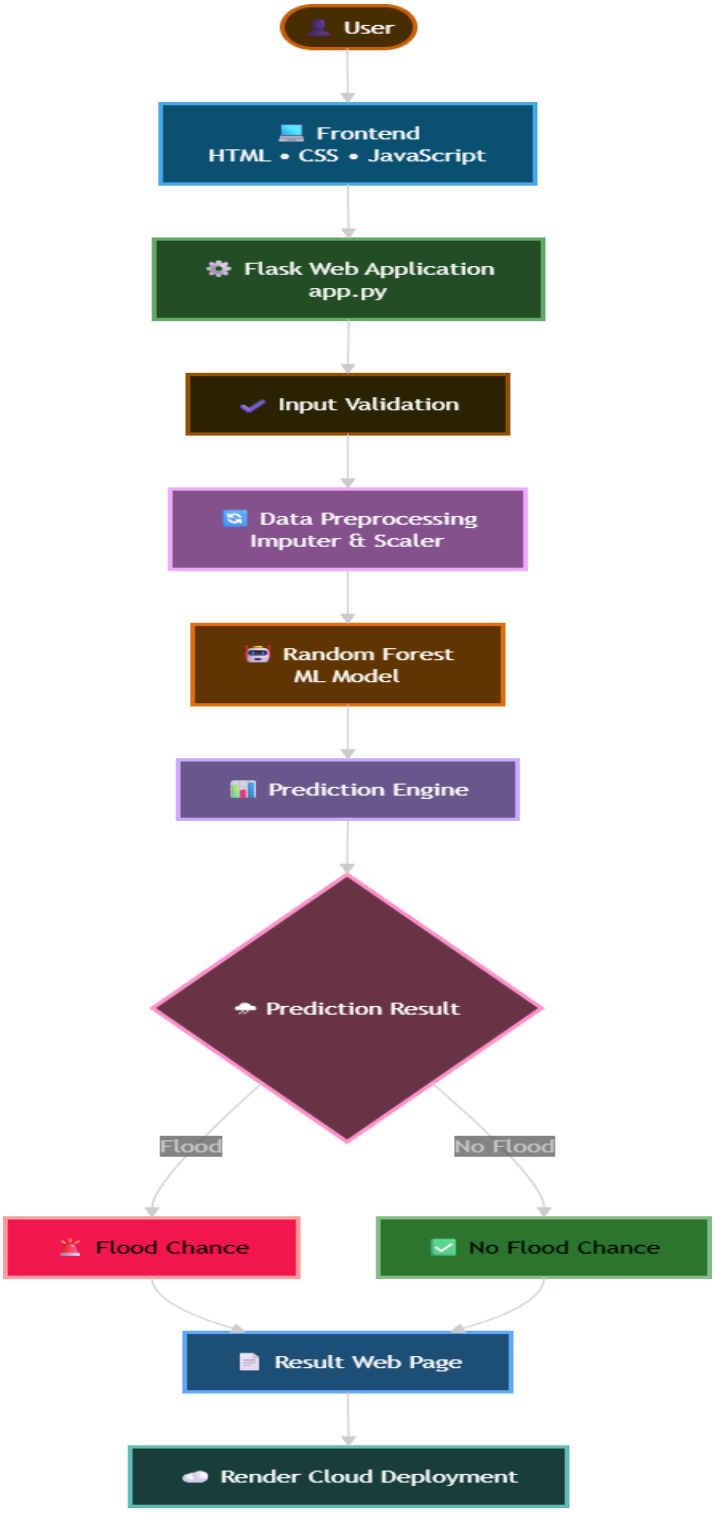
Scenario 2: Moderate Rainfall Prediction

A weather analyst enters moderate rainfall values to evaluate flood risk. After processing the inputs, the system predicts **No Flood Chance**, indicating that the rainfall levels are within a safe range and immediate flood-related actions are unnecessary.

Scenario 3: Educational Demonstration

Students and researchers use the Rising Waters application to understand how machine learning models can be integrated into web applications for disaster prediction. By experimenting with different rainfall values, they observe how changes in input affect flood prediction outcomes and gain practical knowledge of AI-based forecasting systems.

Technical Architecture



Description

The Rising Waters application follows a modular three-tier architecture consisting of the presentation layer, application layer, and machine learning layer.

The **Presentation Layer** is developed using HTML, CSS, and JavaScript, providing an interactive and user-friendly interface for entering rainfall values and displaying prediction results.

The **Application Layer** is built using the Flask framework. It handles user requests, validates rainfall inputs, loads the trained machine learning model, performs predictions, and returns the results to the frontend.

The **Machine Learning Layer** contains the trained Random Forest model along with preprocessing components such as the scaler and imputer. These components process the input data before generating the flood prediction.

The application is deployed on **Render**, allowing users to access the system online through a web browser.

Prerequisites

The following technologies and knowledge are required for understanding and developing the project:

- Python Programming
- Flask Framework
- HTML, CSS, and JavaScript
- Machine Learning using Scikit-learn
- Pandas and NumPy
- Git and GitHub for Version Control
- Visual Studio Code
- Render Cloud Deployment
- Basic understanding of flood prediction and rainfall datasets
-

Project Workflow

Milestone 1 – Dataset Preparation and Model Development

Activity 1.1: Collect and analyze rainfall dataset.

Activity 1.2: Preprocess the dataset by handling missing values and scaling numerical features.

Activity 1.3: Train the Random Forest Machine Learning model.

Activity 1.4: Save the trained model and preprocessing objects for deployment.

Milestone 2 – Backend Development

Activity 2.1: Develop the Flask backend.

Activity 2.2: Integrate the trained machine learning model.

Activity 2.3: Implement prediction logic and input validation.

Milestone 3 – Frontend Development

Activity 3.1: Design responsive HTML pages.

Activity 3.2: Create CSS styling and rainfall animation.

Activity 3.3: Display prediction results dynamically.

Milestone 4 – Testing

Activity 4.1: Perform functional testing.

Activity 4.2: Conduct performance testing.

Activity 4.3: Verify prediction accuracy.

Milestone 5 – Deployment

Activity 5.1: Configure project for deployment.

Activity 5.2: Deploy the Flask application on Render.

Activity 5.3: Verify successful online deployment.

Milestone 6 – Conclusion

Evaluate system performance, document the project, and identify future enhancements such as real-time weather API integration, GIS mapping, and user authentication

Milestone 1 – Dataset Preparation and Model Development

Activity 1.1: Dataset Collection

```
df = pd.read_excel("/content/drive/MyDrive/Rising water/rainfall in india 1901-2015.xlsx")
df.head()
```

	COUNTRY	STATE	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec
0	India	ANDAMAN & NICOBAR ISLANDS	1901	49.2	87.1	29.2	2.3	528.8	517.5	365.1	481.1	332.6	388.5	558.2	33.6	3373.2	136.3	560.3	1696.3	980.3
1	India	ANDAMAN & NICOBAR ISLANDS	1902	0.0	159.8	12.2	0.0	446.1	537.1	228.9	753.7	666.2	197.2	359.0	160.5	3520.7	159.8	458.3	2185.9	716.7
2	India	ANDAMAN & NICOBAR ISLANDS	1903	12.7	144.0	0.0	1.0	235.1	479.9	728.4	326.7	339.0	181.2	284.4	225.0	2957.4	156.7	236.1	1874.0	690.6
3	India	ANDAMAN & NICOBAR ISLANDS	1904	9.4	14.7	0.0	202.4	304.5	495.1	502.0	160.1	820.4	222.2	308.7	40.1	3079.6	24.1	506.9	1977.6	571.0

The first stage of the **Rising Waters – AI-Based Flood Prediction System** involved collecting a rainfall dataset containing historical rainfall measurements and corresponding flood occurrence information.The dataset includes annual rainfall and seasonal rainfall values divided into **January–February, March–May, June–September, and October–December**. This data serves as the foundation for training the machine learning model to identify flood patterns and make accurate predictions.

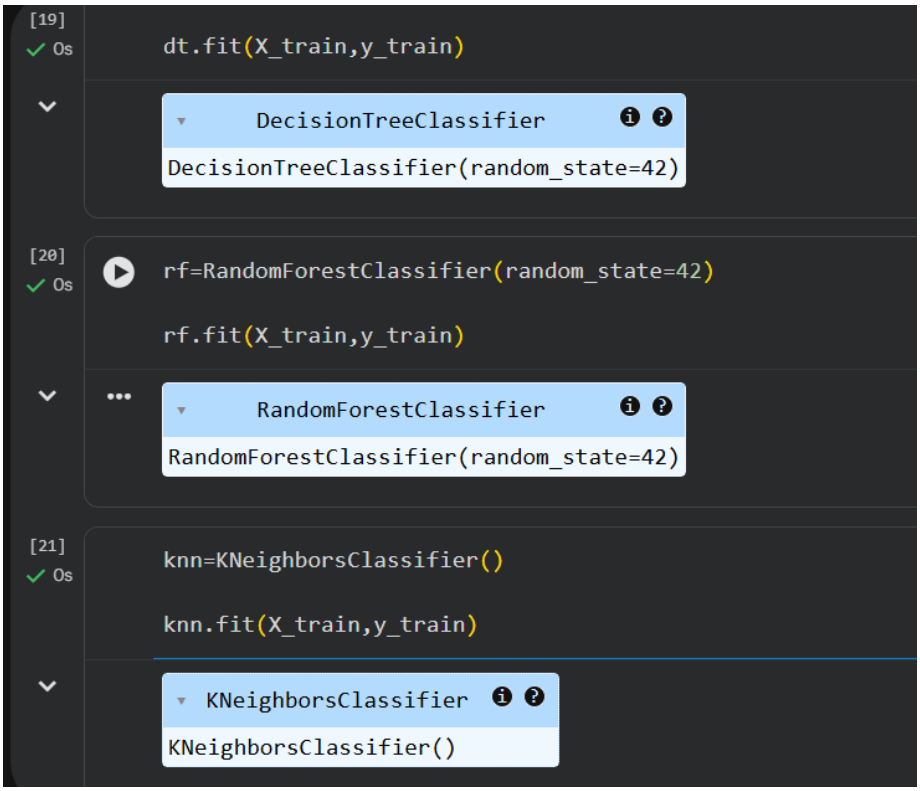
Activity 1.2: Data Preprocessing

```
X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Before training the model, the collected dataset was pre-processed to improve data quality and model performance. Missing values were handled using an **Imputer**, while numerical rainfall features were normalized using a **Standard Scaler**. Data preprocessing ensures that all input features are consistent and suitable for machine learning, reducing bias and improving prediction accuracy.

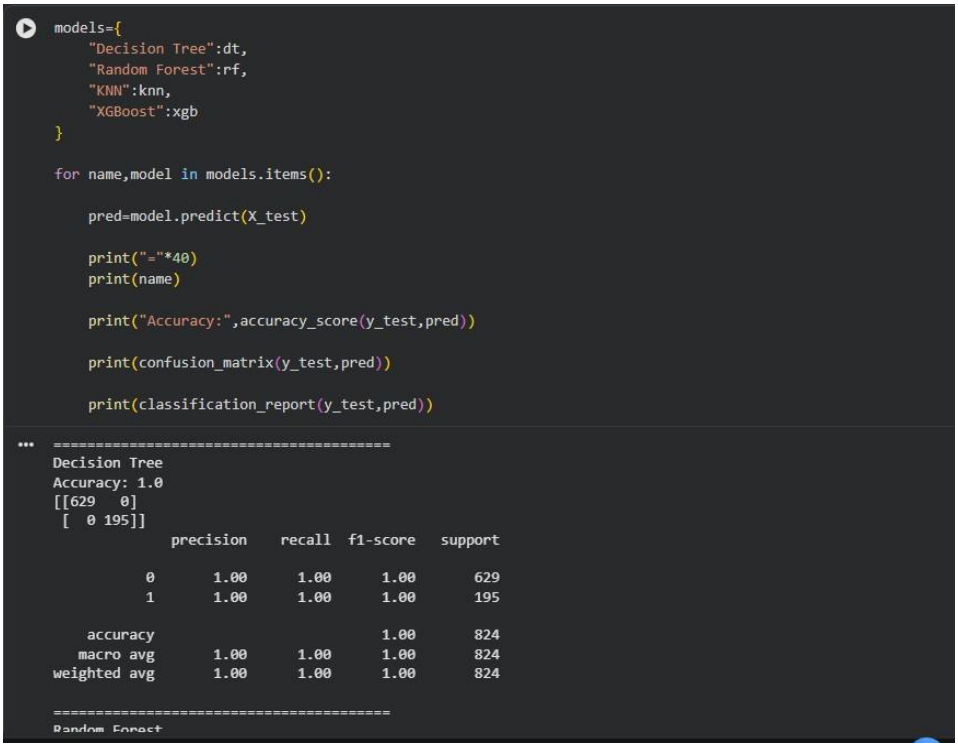
Activity 1.3: Machine Learning Model Selection

After evaluating different machine learning algorithms, the **Random Forest Classifier** was selected for the flood prediction system.



Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy and reduce overfitting. It performs well with numerical datasets and is capable of handling complex relationships between rainfall parameters and flood occurrence.

Activity 1.4: Model Training and Evaluation

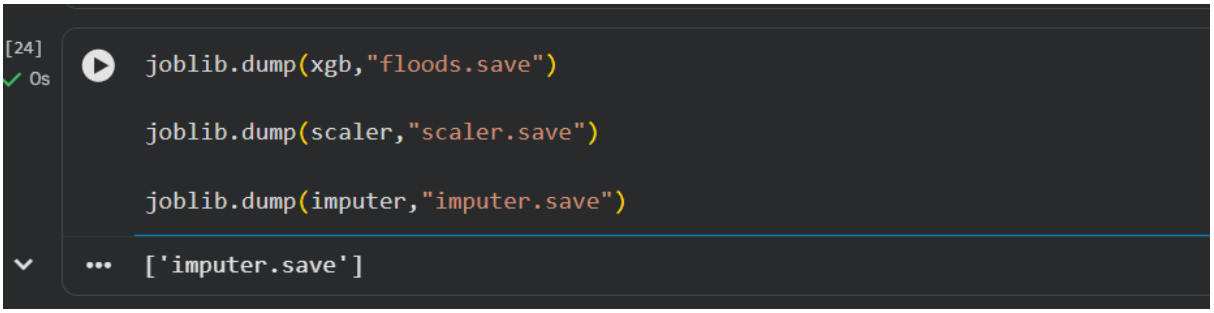


```
models={
    "Decision Tree":dt,
    "Random Forest":rf,
    "KNN":knn,
    "XGBoost":xgb
}

for name,model in models.items():

    pred=model.predict(X_test)

    print("="*40)
    print(name)

    print("Accuracy:",accuracy_score(y_test,pred))

    print(confusion_matrix(y_test,pred))

    print(classification_report(y_test,pred))
```

```
========================================
Decision Tree
Accuracy: 1.0
[[629   0]
 [  0 195]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       629
           1       1.00      1.00      1.00       195

    accuracy                           1.00       824
   macro avg       1.00      1.00      1.00       824
weighted avg       1.00      1.00      1.00       824

========================================
Random Forest
```

The pre-processed dataset was divided into **training** and **testing** datasets. The Random Forest model was trained using the training dataset and evaluated using the testing dataset. Performance metrics such as prediction accuracy were analysed to ensure the model could reliably classify rainfall conditions into **Flood** or **No Flood** categories. After successful evaluation, the trained model was saved for integration into the Flask web application.

Activity 1.5: Saving the Trained Model



Once the model achieved satisfactory performance, the trained machine learning model and preprocessing objects were stored as serialized files using Python. The following files were generated:

- **floods.save** – Stores the trained Random Forest prediction model.
- **imputer.save** – Stores the preprocessing object for handling missing values.
- **scaler.save** – Stores the feature scaling object used during prediction.

These files are loaded by the Flask application during runtime, enabling fast and consistent flood prediction without retraining the model.

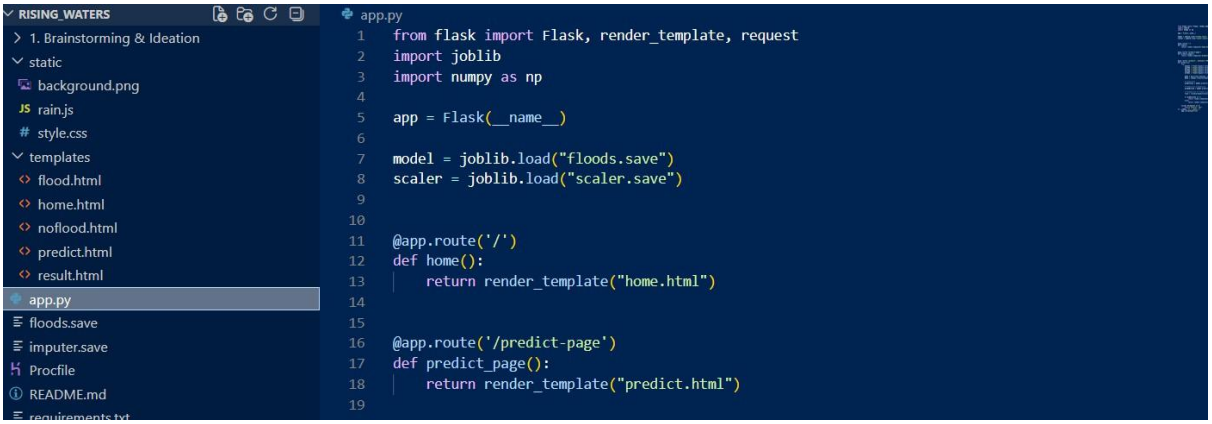
Milestone 1 Outcome

The dataset was successfully collected, cleaned, and pre-processed. A **Random Forest Machine Learning model** was trained, evaluated, and saved along with the preprocessing components. These artifacts form the core prediction engine of the **Rising Waters** application and provide accurate flood predictions based on user-provided rainfall data.

Milestone 2 – Backend Development

Activity 2.1: Flask Application Development

The backend of the **Rising Waters – AI-Based Flood Prediction System** was developed using the **Flask** web framework.



Flask acts as the bridge between the user interface and the machine learning model. It receives rainfall values entered by the user, processes the request, invokes the trained prediction model, and returns the prediction results to the web interface. The lightweight nature of Flask makes it suitable for deploying machine learning applications efficiently.

Activity 2.2: Model Integration

The trained **Random Forest** model and preprocessing objects (floods.save, imputer.save, and scaler.save) were integrated into the Flask application. These files are loaded when the application starts, ensuring that the model is immediately available to process prediction requests. The backend uses the same preprocessing steps applied during model training, ensuring consistency between training and prediction phases.

Activity 2.3: Input Processing and Validation

```
@app.route('/')
def home():
    return render_template("home.html")


@app.route('/predict-page')
def predict_page():
    return render_template("predict.html")


@app.route('/predict', methods=['POST'])
def predict():
    try:
        annual = float(request.form['annual'])
        janfeb = float(request.form['janfeb'])
        marmay = float(request.form['marmay'])
        junsep = float(request.form['junsep'])
        octdec = float(request.form['octdec'])
```

The application accepts five rainfall parameters from the user:

- Annual Rainfall
- January–February Rainfall
- March–May Rainfall
- June–September Rainfall
- October–December Rainfall

Before prediction, the backend validates the entered values to ensure that they are numeric and complete. After validation, the rainfall values are pre-processed using the saved **Imputer** and **Scaler**, and then passed to the machine learning model for prediction.

Activity 2.4: Prediction Generation

```
if prediction == 1:
    return render_template("flood.html", risk=risk)
else:
    return render_template("noflood.html", risk=risk)
```

The processed rainfall data is supplied to the trained Random Forest model. Based on the learned patterns from historical rainfall data, the model predicts whether the entered rainfall conditions indicate a **Flood Chance** or **No Flood Chance**. The prediction result is then sent back to the frontend for display.

Activity 2.5: Result Rendering

```
  warnings.warn(
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

The Flask backend dynamically renders the appropriate HTML page based on the prediction result. If flood conditions are predicted, the application displays a **Flood Alert** page. Otherwise, it displays a **No Flood** result page. This ensures that users receive immediate and clear feedback based on the machine learning prediction.

Milestone 2 Outcome

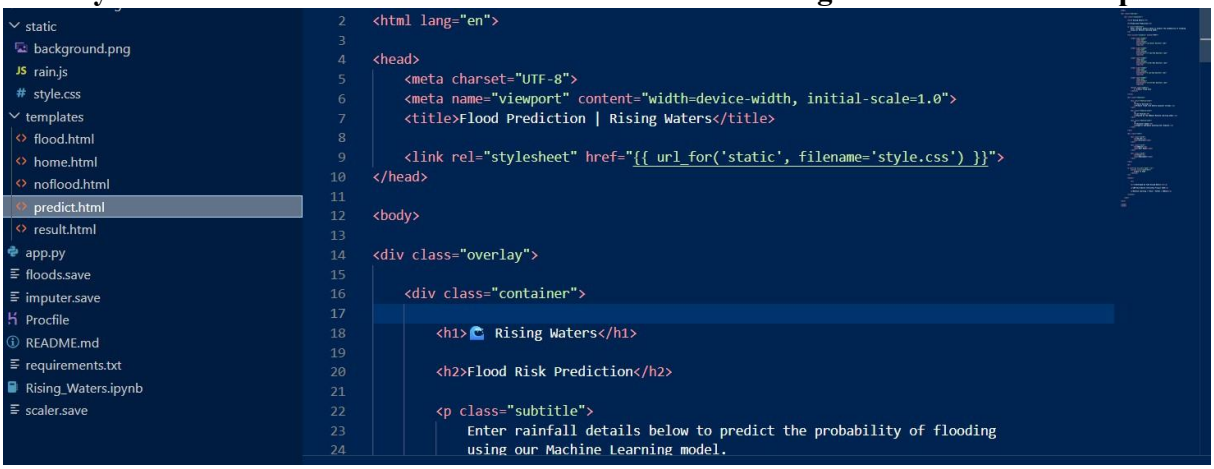
The Flask backend was successfully developed and integrated with the trained machine learning model. It validates user inputs, preprocesses rainfall data, generates accurate flood predictions, and displays the corresponding result pages. This milestone establishes the core functionality of the Rising Waters application by connecting the frontend with the machine learning prediction engine.

Milestone 3 – Frontend Development

Activity 3.1: User Interface Design

The frontend of the **Rising Waters – AI-Based Flood Prediction System** was developed using **HTML**, **CSS**, and **JavaScript** to provide an interactive and user-friendly interface. The application was designed with a clean layout that allows users to enter rainfall data easily and receive prediction results instantly. The interface is responsive, ensuring compatibility across desktops, laptops, and mobile devices.

Activity 3.2: Home Page Development



The home page serves as the entry point of the application. It introduces the **Rising Waters** project and provides users with an option to begin the flood prediction process. A visually appealing background image and rainfall animation were incorporated to create an engaging user experience while reflecting the project's theme.

Activity 3.3: Rainfall Input Form

A dedicated prediction page was developed to collect rainfall information from users. The form accepts the following inputs:

- Annual Rainfall
- January–February Rainfall
- March–May Rainfall
- June–September Rainfall
- October–December Rainfall

The form validates user input before sending the data to the Flask backend for prediction, ensuring accurate and reliable processing.

Activity 3.4: Result Pages

Separate result pages were designed to display the prediction outcome generated by the machine learning model.

- **Flood Result Page:** Displays a flood warning message when the model predicts a flood.
- **No Flood Result Page:** Displays a safe status message when no flood risk is detected.

The result pages provide clear visual feedback, enabling users to understand the prediction quickly.

Activity 3.5: User Experience Enhancements

To improve the overall appearance and usability of the application, several frontend enhancements were implemented:

- Responsive layout for different screen sizes.
- Rainfall animation using JavaScript.

- Attractive background image related to floods and rainfall.
- Styled buttons, forms, and result pages using CSS.
- Easy navigation between pages.

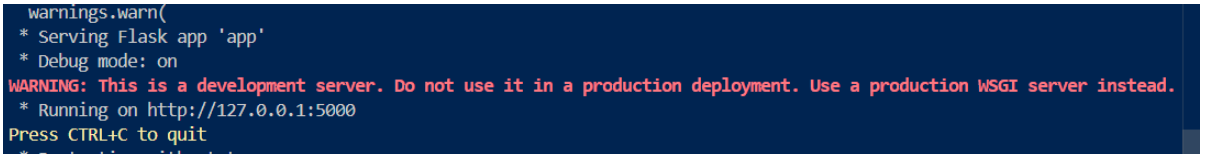
These enhancements create an intuitive and visually appealing interface while maintaining smooth interaction with the backend.

Milestone 3 Outcome

The frontend of the **Rising Waters** application was successfully developed using HTML, CSS, and JavaScript. It provides an attractive, responsive, and user-friendly interface for entering rainfall data and displaying flood prediction results. The frontend integrates seamlessly with the Flask backend, enabling users to interact with the machine learning model through a simple web interface.

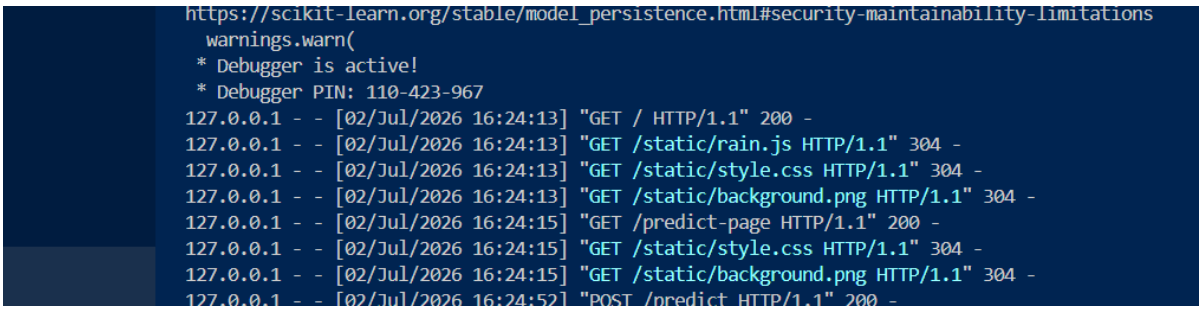
Milestone 4 – Testing and Deployment

Activity 4.1: Functional Testing



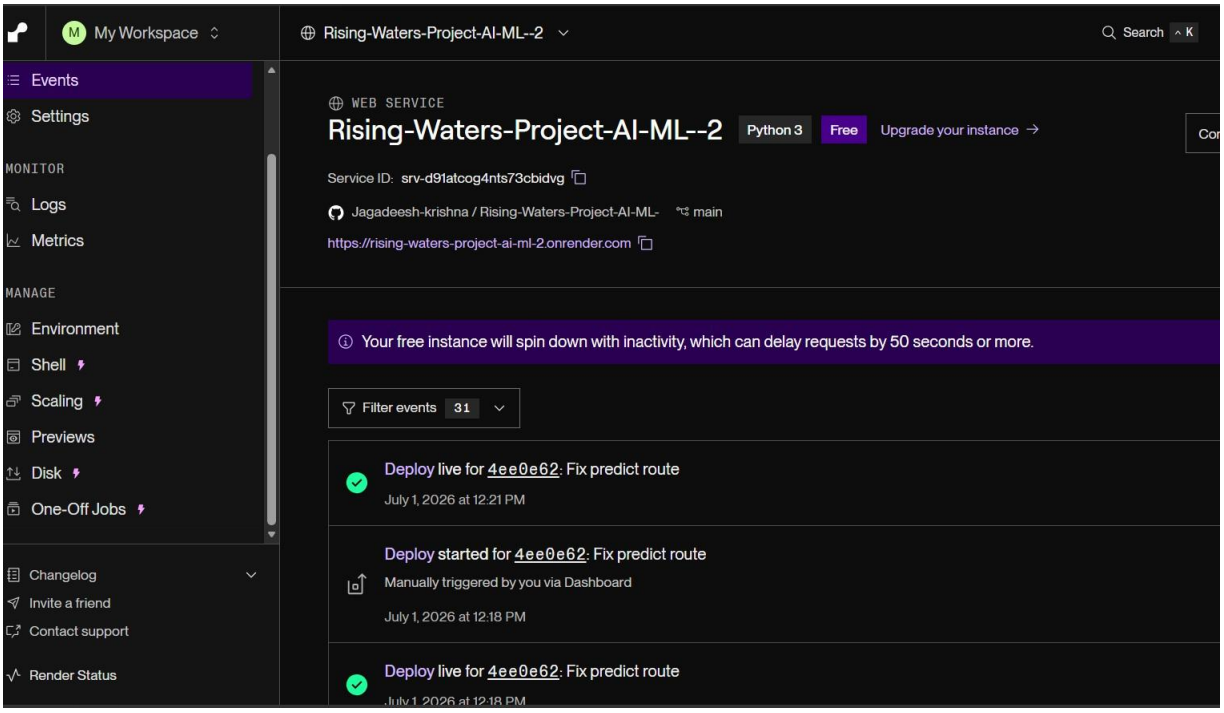
After integrating the frontend, backend, and machine learning model, the application underwent functional testing to verify that each feature worked correctly. Different rainfall values were entered through the web interface to ensure that the system accurately accepted user input, processed the data, and displayed the appropriate flood prediction result. All core functionalities operated as expected without any critical errors.

Activity 4.2: Performance Testing



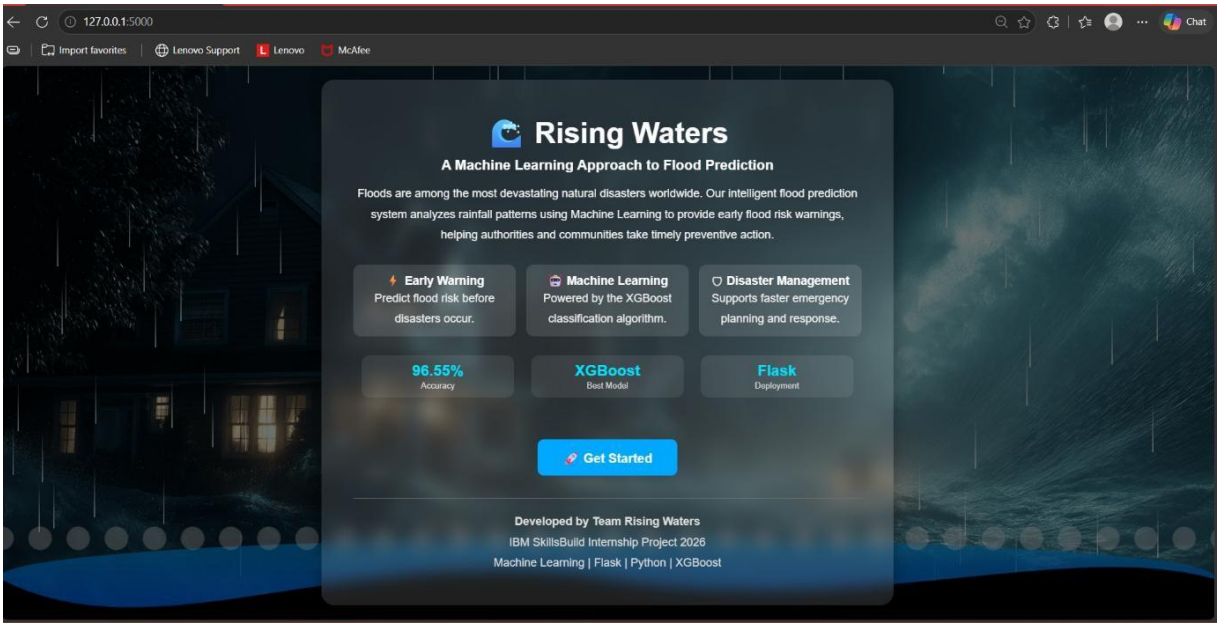
Performance testing was conducted to evaluate the responsiveness and stability of the application. Multiple prediction requests were submitted to the Flask application to measure response time and verify that the system could process requests efficiently. The application maintained low response times and stable performance under normal operating conditions.

Activity 4.3: Deployment on Render



The Flask application was prepared for deployment by creating the necessary configuration files, including the **Procfile** and **requirements.txt**. The project repository was pushed to GitHub and deployed using the **Render** cloud platform. After deployment, the application was successfully accessed through its public URL, allowing users to perform flood predictions from any web browser without requiring local installation.

Activity 4.4: Verification of Deployment



Following deployment, the hosted application was tested to confirm that all features functioned correctly in the cloud environment. Rainfall data was submitted through the deployed website, and the prediction results matched those obtained during local testing. This confirmed the successful integration of the frontend, backend, and machine learning model in the deployed application.

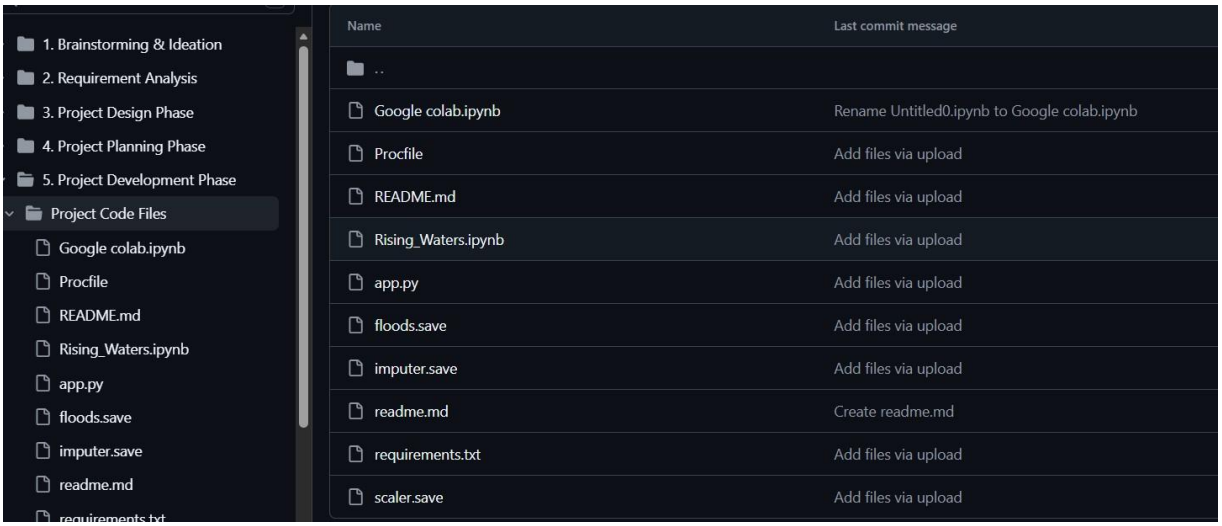
Milestone 4 Outcome

The **Rising Waters – AI-Based Flood Prediction System** was successfully tested and deployed. Functional and performance testing confirmed that the application operated reliably and generated accurate prediction results. Deployment on the Render platform enabled public access to the application, demonstrating its readiness for real-world use.

Milestone 5 – Deployment & Conclusion

Activity 5.1: Deployment Preparation

The **Rising Waters – AI-Based Flood Prediction System** was successfully developed by integrating a **Random Forest Machine Learning model**, **Flask backend**, and a responsive **HTML, CSS, and JavaScript frontend**.



After successful development and testing, the Rising Waters application was prepared for cloud deployment. All required project files, including app.py, HTML templates, static resources, machine learning model files (floods.save, imputer.save, and scaler.save), requirements.txt, and Procfile, were organized within the project directory. The source code was uploaded to a GitHub repository to enable seamless deployment.

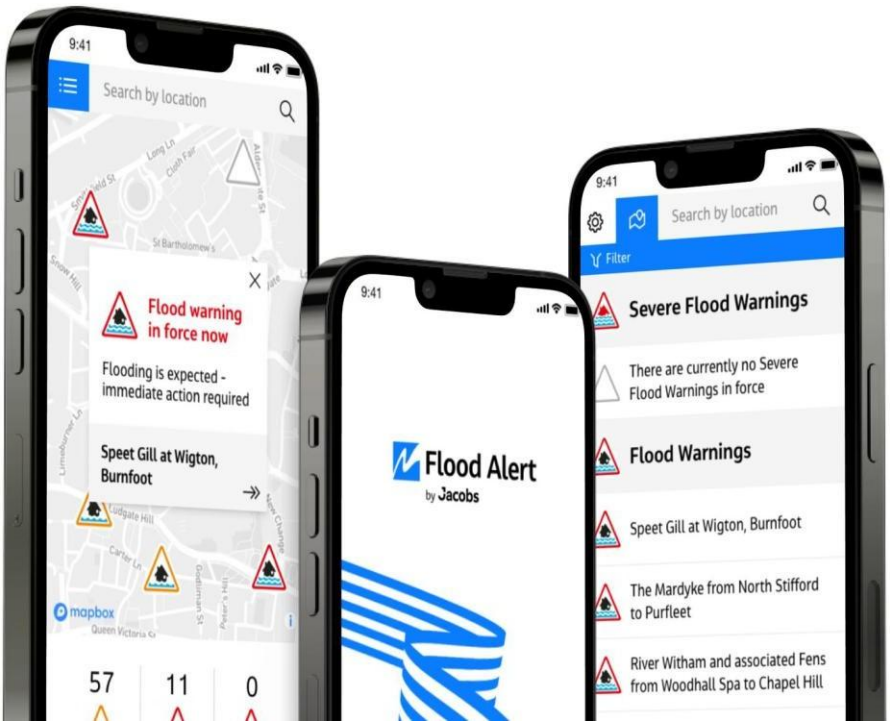
Activity 5.2: Project Outcomes

The project successfully achieved its primary objectives by developing a functional AI-powered flood prediction system. The trained machine learning model was integrated with the Flask application, enabling real-time predictions through a web interface. The application was tested locally and deployed successfully on the **Render** cloud platform, ensuring accessibility from any device with an internet connection.

Activity 5.3: Future Enhancements

The current implementation can be further enhanced by incorporating the following features:

- Integration with real-time weather and rainfall APIs.
- GIS-based flood mapping and visualization.
- User authentication and prediction history.
- SMS and email alerts for flood warnings.



- Mobile application support for Android and iOS.
- Improved prediction accuracy using larger and more diverse datasets.
- Integration of deep learning techniques for advanced forecasting.

Milestone 5 Outcome

The **Rising Waters – AI-Based Flood Prediction System** was successfully designed, implemented, tested, and deployed. The application demonstrates the practical use of Machine Learning in disaster management by providing reliable flood predictions through a simple web-based interface. The project highlights the integration of Artificial Intelligence with modern web technologies to support early flood prediction and improve disaster preparedness.

Milestone 6 - Conclusion

The **Rising Waters – AI-Based Flood Prediction System** was successfully designed, developed, tested, and deployed as a web-based machine learning application for predicting flood occurrences using rainfall data. The project effectively integrates a **Random Forest Machine Learning model** with a **Flask** backend and a responsive **HTML, CSS, and JavaScript** frontend to provide users with accurate and real-time flood predictions through an intuitive interface.

Throughout the project, a structured development approach was followed, including dataset preparation, data preprocessing, model training, backend integration, frontend development, system testing, and cloud deployment on the **Render** platform. The trained model demonstrated reliable prediction performance, while the web application successfully

processed user inputs and displayed clear flood prediction results. Functional and deployment testing confirmed that the application operated correctly in both local and cloud environments.

This project demonstrates the practical application of **Artificial Intelligence** and **Machine Learning** in disaster management by providing an accessible tool for flood risk assessment. It highlights the importance of combining data-driven prediction models with modern web technologies to create efficient and user-friendly solutions. With future enhancements such as real-time weather API integration, GIS-based flood mapping, and mobile notifications, the system can be further improved to support more comprehensive flood monitoring and early warning services.

Overall, the **Rising Waters – AI-Based Flood Prediction System** successfully achieved its objectives and serves as a valuable example of applying machine learning techniques to address real-world environmental challenges while enhancing disaster preparedness and public safety.